

Schlüsselwort/Typ	Kategorie	Beschreibung
int	primitiver Datentyp	Ganzzahl (32 Bit)
double	primitiver Datentyp	Gleitkommazahl (64 Bit)
boolean	primitiver Datentyp	Wahrheitswert (true/false)
char	primitiver Datentyp	Zeichen (16 Bit)
String	Klasse	Zeichenkette
public	Modifizierer	Öffentlich sichtbar
private	Modifizierer	Nur in der Klasse sichtbar
protected	Modifizierer	Im Paket und Subklassen sichtbar
static	Modifizierer	Gehört zur Klasse, nicht zur Instanz
final	Modifizierer	Nicht überschreibbar/veränderbar
abstract	Modifizierer	Abstrakte Klasse/Methode
interface	Schlüsselwort	Schnittstelle
extends	Schlüsselwort	Vererbung von Klassen
implements	Schlüsselwort	Implementiert Interface
new	Schlüsselwort	Erzeugt neues Objekt
this	Schlüsselwort	Referenz auf aktuelles Objekt
super	Schlüsselwort	Referenz auf Oberklasse

Erklärung: Diese Tabelle listet grundlegende Java-Schlüsselwörter und Datentypen auf, die für die Programmierung in Java essentiell sind. Die Kategorie gibt an, ob es sich um einen Datentyp, Modifizierer oder ein Schlüsselwort handelt. Die Beschreibung erläutert die Funktion, z. B. dass `int` Ganzzahlen repräsentiert oder `public` die Sichtbarkeit einer Klasse definiert. Diese Begriffe sind zentral für das Verständnis der Syntax und Struktur von Java-Programmen.

Modifizierer	Wirkung	Verwendung
public	überall sichtbar	Klasse, Methode, Feld
private	nur in der Klasse sichtbar	Methode, Feld
protected	im Paket und Subklassen sichtbar	Methode, Feld
default (kein Schlüsselwort)	nur im Paket sichtbar	Klasse, Methode, Feld
static	gehört zur Klasse	Methode, Feld
final	nicht überschreibbar	Klasse, Methode, Feld, Variable
abstract	muss überschrieben werden	Klasse, Methode
synchronized	thread-sicher	Methode, Block
volatile	variable kann sich jederzeit ändern	Feld
transient	wird nicht serialisiert	Feld

Erklärung: Diese Tabelle beschreibt die Modifizierer in Java, die den Zugriff und das Verhalten von Klassen, Methoden und Feldern steuern. Beispielsweise sorgt `public` für allgemeine Sichtbarkeit, während `final` Änderungen verhindert. Die Tabelle zeigt, wo jeder Modifizierer angewendet werden kann und welche Wirkung er hat.

Operator	Typ	Beispiel/Bedeutung
+	arithmetisch	a + b (Addition)
-	arithmetisch	a - b (Subtraktion)
*	arithmetisch	a * b (Multiplikation)
/	arithmetisch	a / b (Division)
%	arithmetisch	a % b (Rest)
++	Inkrement	a++ (erhöht um 1)
--	Dekrement	a-- (verringert um 1)
==	Vergleich	a == b (gleich?)
!=	Vergleich	a != b (ungleich?)
>	Vergleich	a > b (größer?)
<	Vergleich	a < b (kleiner?)
>=	Vergleich	a >= b (größer gleich?)
<=	Vergleich	a <= b (kleiner gleich?)
&&	logisch	a && b (UND)
	logisch	a b (ODER)
!	logisch	! a (NICHT)
=	Zuweisung	a = b
+=	Zuweisung	a += b (a = a + b)
-=	Zuweisung	a -= b (a = a - b)
instanceof	Typprüfung	obj instanceof String

Erklärung: Diese Tabelle fasst die wichtigsten Operatoren in Java zusammen, einschließlich arithmetischer, logischer, Vergleichs- und Zuweisungsoperatoren. Die Beispiele verdeutlichen die Anwendung, z. B. wie ++ eine Variable inkrementiert oder && logische Bedingungen kombiniert.

Collection-Typ	Interface/Klasse	Eigenschaften	Besonderheiten
List	Interface	geordnete Sammlung, erlaubt Duplikate	Zugriff über Index, z.B. ArrayList, LinkedList
ArrayList	Klasse	schnelle Suche, langsames Einfügen/Löschen	basiert auf Array, dynamisch erweiterbar
LinkedList	Klasse	schnelles Einfügen/Löschen	doppelt verkettete Liste, auch Queue
Set	Interface	keine Duplikate, ungeordnet	z.B. HashSet, TreeSet
HashSet	Klasse	schnelle Suche, keine Reihenfolge	basiert auf HashMap
TreeSet	Klasse	sortiert, keine Duplikate	basiert auf TreeMap, natürliche Ordnung
Queue	Interface	Warteschlange (FIFO)	z.B. LinkedList, PriorityQueue
PriorityQueue	Klasse	sortierte Queue	Elemente nach Priorität geordnet
Map	Interface	Schlüssel-Wert-Paare	keine Duplikate bei Schlüsseln, z.B. HashMap, TreeMap
HashMap	Klasse	schnelle Suche, keine Reihenfolge	erlaubt null-Schlüssel und -Werte
TreeMap	Klasse	sortierte Schlüssel	natürliche Ordnung, keine null-Schlüssel
LinkedHashMap	Klasse	Einfügereihenfolge bleibt erhalten	kombiniert HashMap mit Liste

Erklärung: Diese Tabelle gibt einen Überblick über die Java Collections Framework-Klassen und -Interfaces. Sie beschreibt Eigenschaften wie die Reihenfolge oder Duplikatbehandlung und Besonderheiten wie die Geschwindigkeit von ArrayList bei der Suche oder die Sortierung in TreeSet.

Methoden/Operator	Typ	Beschreibung	Beispiel
filter	Zwischenoperation	Filtert nach Bedingung	<code>stream.filter(x -> x > 0)</code>
map	Zwischenoperation	Wendet Funktion auf jedes Element an	<code>stream.map(String::length)</code>
flatMap	Zwischenoperation	”Entpackt” verschachtelte Strukturen	<code>stream.flatMap(Collection::stream)</code>
sorted	Zwischenoperation	Sortiert die Elemente	<code>stream.sorted()</code>
distinct	Zwischenoperation	Entfernt Duplikate	<code>stream.distinct()</code>
limit	Zwischenoperation	Beschränkt Anzahl der Elemente	<code>stream.limit(10)</code>
skip	Zwischenoperation	Überspringt n Elemente	<code>stream.skip(5)</code>
forEach	Terminaloperation	Führt Aktion für jedes Element aus	<code>stream.forEach(System.out::println)</code>
collect	Terminaloperation	Sammelt Ergebnis in Collection	<code>stream.collect(Collectors.toList())</code>
reduce	Terminaloperation	Reduziert auf einen Wert	<code>stream.reduce(0, Integer::sum)</code>
count	Terminaloperation	Zählt Elemente	<code>stream.count()</code>
anyMatch	Terminaloperation	Prüft, ob Bedingung für ein Element gilt	<code>stream.anyMatch(x -> x == 5)</code>
allMatch	Terminaloperation	Prüft, ob Bedingung für alle gilt	<code>stream.allMatch(x -> x > 0)</code>
noneMatch	Terminaloperation	Prüft, ob kein Element Bedingung erfüllt	<code>stream.noneMatch(x -> x < 0)</code>
findFirst	Terminaloperation	Erstes Element finden	<code>stream.findFirst()</code>
findAny	Terminaloperation	Irgendein Element finden	<code>stream.findAny()</code>

Erklärung: Diese Tabelle erläutert die wichtigsten Stream-Operationen in Java, die für die funktionale Verarbeitung von Daten verwendet werden. Zwischenoperationen wie `filter` oder `map` verändern den Stream, während Terminaloperationen wie `collect` das Endergebnis liefern.

Thema	Syntax/Beispiel	Beschreibung
Deklaration 1D-Array	<code>double[] arr;</code>	Array von double-Werten deklarieren
Initialisierung 1D-Array	<code>arr = new double[5];</code>	Array mit 5 Elementen erstellen
Direkte Initialisierung	<code>double[] arr = {1.2, 3.4, 5.6};</code>	Werte direkt zuweisen
Zugriff auf Element	<code>arr[0] = 2.5;</code>	Erstes Element setzen
Länge abfragen	<code>arr.length</code>	Anzahl der Elemente im Array
Schleife über 1D-Array	<code>for(int i=0; i<arr.length; i++) ...</code>	Zugriff auf alle Elemente
Deklaration 2D-Array	<code>double[][] matrix;</code>	2D-Array deklarieren
Initialisierung 2D-Array	<code>matrix = new double[3][4];</code>	3 Zeilen, 4 Spalten
Direkte Initialisierung 2D	<code>double[][] m = {{1,2},{3,4}};</code>	2x2-Matrix mit Werten
Zugriff auf Element 2D	<code>matrix[1][2] = 5.0;</code>	2. Zeile, 3. Spalte setzen
Länge Zeilen	<code>matrix.length</code>	Anzahl der Zeilen
Länge Spalten	<code>matrix[0].length</code>	Spalten der ersten Zeile
Schleife über 2D-Array	<code>for(int i=0; i<m.length; i++) for(int j=0; j<m[i].length; j++) ...</code>	Über alle Einträge iterieren

Erklärung: Diese Tabelle beschreibt die Arbeit mit Arrays in Java, einschließlich Deklaration, Initialisierung und Zugriff. Sie behandelt sowohl ein- als auch zweidimensionale Arrays und zeigt, wie Schleifen verwendet werden, um Arrays zu durchlaufen.

Typumwandlung	Syntax/Beispiel	Beschreibung
Implizite Umwandlung (Widening)	<code>int i = 5; double d = i;</code>	Automatisch, kein Datenverlust
Explizite Umwandlung (Narrowing)	<code>double d = 3.7; int i = (int) d;</code>	Explizit, evtl. Datenverlust
Primitive zu String	<code>String s = String.valueOf(zahl);</code>	Zahl in String umwandeln
String zu int	<code>int i = Integer.parseInt(s);</code>	String in int umwandeln
String zu double	<code>double d = Double.parseDouble(s);</code>	String in double umwandeln
Objekt zu Subtyp (Downcast)	<code>Tier t = new Hund(); Hund h = (Hund) t;</code>	Cast von Oberklasse zu Unterklasse
Subtyp zu Obertyp (Upcast)	<code>Hund h = new Hund(); Tier t = h;</code>	Automatisch, keine Angabe nötig
Objekt zu String	<code>String s = obj.toString();</code>	Objekt als String darstellen
char zu int	<code>char c = 'A'; int i = c;</code>	ASCII/Unicode-Wert des Zeichens
int zu char	<code>int i = 65; char c = (char) i;</code>	Zahl als Zeichen interpretieren
char zu String	<code>char c = 'A'; String s = Character.toString(c);</code>	Einzelnes Zeichen zu String
String zu char[]	<code>char[] arr = s.toCharArray();</code>	String in char-Array umwandeln
char[] zu String	<code>String s = new String(arr);</code>	char-Array zu String

Erklärung: Diese Tabelle erklärt Typumwandlungen in Java, sowohl implizite (automatische) als auch explizite (manuelle). Sie umfasst Umwandlungen zwischen primitiven Typen, Strings und Objekten sowie Beispiele für Up- und Downcasting.

Kurzschriftweise	Bedeutung	Beispiel
<code>a = (b < c) ? b : c;</code>	Ternärer Operator (if-else als Ausdruck)	Setzt a auf b, wenn b < c, sonst auf c
<code>for (int x : liste)</code>	For-each-Schleife	Durchläuft alle Elemente in liste
<code>++i, i++</code>	Inkrement (vor/nach)	Erhöht i um 1 (Prä- und Postfix)
<code>--i, i--</code>	Dekrement (vor/nach)	Verringert i um 1 (Prä- und Postfix)
<code>+=, -=, *=, /=</code>	Kurzformen für Operation und Zuweisung	<code>a += 2;</code> entspricht <code>a = a + 2;</code>
<code>System.out.println</code>	Kurzform für Konsolenausgabe	Gibt Wert auf Konsole aus
<code>import static</code> ...	Statischer Import	Ermöglicht Zugriff auf statische Member ohne Klassennamen

Erklärung: Diese Tabelle zeigt Kurzschriftweisen in Java, die den Code kompakter machen, wie den ternären Operator oder die for-each-Schleife. Sie erläutert die Bedeutung und gibt Beispiele für die praktische Anwendung.

Schleifenart	Syntax/Beispiel	Beschreibung
For-Schleife	<code>for (int i = 0; i < 10; i++) { ... }</code>	Wiederholt für festgelegte Anzahl von Iterationen
While-Schleife	<code>while (bedingung) { ... }</code>	Wiederholt, solange Bedingung wahr ist
Do-While-Schleife	<code>do { ... } while (bedingung);</code>	Führt Schleife mindestens einmal aus
For-Each-Schleife	<code>for(Typ var : collection) { ... }</code>	Iteriert über Elemente einer Sammlung oder Arrays
Break-Anweisung	<code>break;</code>	Beendet Schleife vorzeitig
Continue-Anweisung	<code>continue;</code>	Überspringt Rest der aktuellen Iteration
Verschachtelte Schleifen	<code>for (int i = 0; i < n; i++) for (int j = 0; j < m; j++) { ... }</code>	Schleifen in Schleifen für komplexe Iterationen
Label mit Break	<code>outer: for (...) { break outer; }</code>	Beendet benannte äußere Schleife
Label mit Continue	<code>outer: for (...) { continue outer; }</code>	Überspringt zur nächsten Iteration der äußeren Schleife

Erklärung: Diese Tabelle beschreibt die verschiedenen Schleifenarten in Java, einschließlich `for`, `while` und `do-while`. Sie erklärt auch Steueranweisungen wie `break` und `continue` sowie die Verwendung von Labels.

Element	Groß/Klein	Zwingend?	Beispiel
Variable	camelCase (klein)	Konvention	anzahlDerTage
Methode	camelCase (klein)	Konvention	berechneSumme()
Klasse	PascalCase (groß)	Konvention	RechnungsManager
Konstruktor	PascalCase (groß)	Muss wie Klasse	RechnungsManager()
Konstante	GROSS.UND.UNTERSTRICH	Konvention	MAX_WERT
Interface	PascalCase (groß)	Konvention	Bezahlbar
Paket	klein	Konvention	de.meinprojekt.util
Enum	PascalCase (groß)	Konvention	Wochentag
Enum-Konstante	GROSS.UND.UNTERSTRICH	Konvention	MONTAG
Typ-Parameter (Generic)	Großbuchstabe	Konvention	E, T, K, V
Annotation	PascalCase (groß)	Konvention	@Override
Parameter (Übergabe)	camelCase (klein)	Konvention	wert, summe
Feld (Klassenvariable)	camelCase (klein)	Konvention	gesamtSumme
Objekt	camelCase (klein)	Konvention	kunde, meinAuto
Array	camelCase (klein)	Konvention	namenArray, zahlen
ArrayList	camelCase (klein)	Konvention	produktListe, namenArrayList
Exception-Klasse	PascalCase (groß, ...Exception)	Konvention	DateiNichtGefundenException
Exception-Variable	camelCase (klein)	Konvention	e, ex, exception
Record	PascalCase (groß)	Konvention	PersonRecord
Lambda-Parameter	camelCase (klein)	Konvention	x, eintrag, person

Erklärung: Diese Tabelle beschreibt die Namenskonventionen in Java, die für Lesbarkeit und Konsistenz sorgen. Sie zeigt, wie Variablen, Methoden, Klassen und andere Elemente benannt werden sollten, z. B. camelCase für Variablen oder PascalCase für Klassen.

Konzept	Syntax/Beispiel	Beschreibung
try-Block	<code>try { ... }</code>	Definiert einen Block, in dem Ausnahmen auftreten können
catch-Block	<code>catch (Exception e) { ... }</code>	Behandelt eine spezifische Ausnahme
finally-Block	<code>finally { ... }</code>	Wird immer ausgeführt, unabhängig von Ausnahmen
throw	<code>throw new IOException();</code>	Wirft eine Ausnahme explizit
throws	<code>void methode() throws IOException { ... }</code>	Deklariert, dass eine Methode Ausnahmen werfen kann
Exception	Exception	Basisklasse für alle geprüften Ausnahmen
RuntimeException	RuntimeException	Basisklasse für ungeprüfte Ausnahmen
NullPointerException	<code>obj.toString()</code> (bei <code>obj = null</code>)	Tritt auf, wenn auf null zugegriffen wird
IOException	<code>new FileReader("file.txt")</code>	Tritt bei Ein-/Ausgabe-Fehlern auf
ArrayIndexOutOfBoundsException	<code>arr[10]</code> (bei Array-Länge 5)	Zugriff außerhalb der Array-Grenzen
ArithmeticException	<code>int x = 5 / 0;</code>	Tritt bei ungültigen arithmetischen Operationen auf

Erklärung: Diese Tabelle beschreibt die Mechanismen der Ausnahmebehandlung in Java. Sie umfasst Schlüsselwörter wie `try`, `catch` und `finally` sowie häufige Ausnahmeklassen. Die Beispiele zeigen, wie Ausnahmen ausgelöst und behandelt werden, um robuste Programme zu erstellen.

Konzept	Syntax/Beispiel	Beschreibung
Klassen-Deklaration	<code>class Auto { ... }</code>	Definiert eine Klasse als Bauplan für Objekte
Instanzvariable	<code>int geschwindigkeit;</code>	Variable, die zu jedem Objekt gehört
Konstruktor	<code>Auto(int g) { geschwindigkeit = g; }</code>	Initialisiert ein Objekt beim Erstellen
Methode	<code>void fahre() { ... }</code>	Definiert Verhalten der Klasse
Objekt-Erstellung	<code>Auto meinAuto = new Auto(100);</code>	Erstellt ein Objekt der Klasse
this-Schlüsselwort	<code>this.geschwindigkeit = g;</code>	Verweist auf die aktuelle Instanz
Getter-Methode	<code>int getGeschwindigkeit() { return geschwindigkeit; }</code>	Gibt Wert einer Instanzvariable zurück
Setter-Methode	<code>void setGeschwindigkeit(int g) { geschwindigkeit = g; }</code>	Setzt Wert einer Instanzvariable
toString-Methode	<code>String toString() { return "Auto: " + geschwindigkeit; }</code>	String-Darstellung des Objekts

Erklärung: Diese Tabelle erklärt die Grundlagen von Klassen und Objekten in Java, die das Herzstück der objektorientierten Programmierung bilden. Sie zeigt, wie Klassen definiert, Objekte erstellt und Instanzvariablen sowie Methoden verwendet werden.

Kontrollstruktur	Syntax/Beispiel	Beschreibung
if-Anweisung	<code>if (x < 0) { ... }</code>	Führt Block aus, wenn Bedingung wahr ist
else-Zweig	<code>else { ... }</code>	Führt Block aus, wenn if-Bedingung falsch ist
else if-Zweig	<code>else if (x < 0) { ... }</code>	Prüft zusätzliche Bedingung
switch-Anweisung	<code>switch (x) { case 1: ... break; }</code>	Wählt Block basierend auf Wert von x
case-Zweig	<code>case 1: System.out.println("Eins");</code>	Definiert Aktion für spezifischen Wert
default-Zweig	<code>default: System.out.println("Andere");</code>	Aktion, wenn kein case passt
break in switch	<code>break;</code>	Beendet switch-Block
Ternärer Operator	<code>String s = (x < 0) ? "positiv" : "negativ";</code>	Kompakte if-else-Alternative

Erklärung: Diese Tabelle beschreibt die Kontrollstrukturen in Java, die die Ablaufsteuerung eines Programms ermöglichen. Sie umfasst bedingte Anweisungen wie `if` und `switch` sowie den ternären Operator für kompakte Entscheidungen.

Klasse/Methode	Syntax/Beispiel	Beschreibung
File-Klasse	<code>File file = new File("datei.txt");</code>	Repräsentiert eine Datei oder ein Verzeichnis
FileReader	<code>FileReader fr = new FileReader(file);</code>	Liest Zeichen aus einer Datei
BufferedReader	<code>BufferedReader br = new BufferedReader(fr);</code>	Effizientes Lesen von Textzeilen
<code>readLine()</code>	<code>String line = br.readLine();</code>	Liest eine Zeile aus der Datei
FileWriter	<code>FileWriter fw = new FileWriter("datei.txt");</code>	Schreibt Zeichen in eine Datei
BufferedWriter	<code>BufferedWriter bw = new BufferedWriter(fw);</code>	Effizientes Schreiben von Text
<code>write()</code>	<code>bw.write("Text");</code>	Schreibt Text in die Datei
<code>close()</code>	<code>bw.close();</code>	Schließt die Datei und gibt Ressourcen frei
<code>Files.readAllLines</code>	<code>List<String> lines = Files.readAllLines(Paths.get("datei.txt"));</code>	Liest alle Zeilen einer Datei
<code>Files.write</code>	<code>Files.write(Paths.get("datei.txt"), "Text".getBytes());</code>	Schreibt Text in eine Datei

Erklärung: Diese Tabelle beschreibt die wichtigsten Klassen und Methoden für die Datei- und Ein-/Ausgabeoperationen in Java. Sie zeigt, wie Dateien gelesen und geschrieben werden, mit Fokus auf effiziente Klassen wie `BufferedReader` und `BufferedWriter`.